

Bitstream Evolution: an Open-Source FPGA Intrinsic Evolvable Hardware Toolkit

ALLYN LOYD^{1,2}, DANIEL GAULL^{1,3}, LOGAN MANTHEY^{1,4}, NICKLAS CARPENTER⁵, DEREK WHITLEY^{1,4}, (Member, IEEE), and JASON A. YODER¹, (Member, IEEE)

¹Rose-Hulman Institute of Technology, Terre Haute, IN 47803 USA

²University of Houston, Houston, TX 77204 USA

³Tesla, Inc. Austin, TX 78725 USA

⁴Vivum AI, Bloomington, IN, 47401 USA

⁵University of Arizona, Tucson, Arizona 85721 USA

Corresponding author: Jason A. Yoder (e-mail: yoder1@rose-hulman.edu).

ABSTRACT The field of intrinsic, bitstream-level, evolvable hardware has recently been resurrected and interest in the area has begun to grow. This paper presents a low barrier-to-entry software and hardware platform allowing evolvable hardware researchers to conduct experiments, visualize results, and share data. The system's open-source software is designed to enable iterative enhancements so that contributors from around the world can participate. To encourage community development, we detail the process and results of several of our experiments, which can be used as a reference for further study. Additionally, we discuss the results of our experiments, focusing in particular on their implications with respect to the question of robustness and transferability of evolved hardware circuits. Our results demonstrate significant potential for evolving circuits that not only maintain long-term functionality but can also operate effectively outside their original evolutionary environments. These results demonstrate the feasibility of intrinsic, bitstream level evolution which, in the long run, could increase the efficiency and lower the power requirements of modern computing and artificial intelligence systems. We discuss open questions for the field and directions for future work including stability, transferability, robustness, quality diversity, signal processing, and security.

INDEX TERMS analog computing, bitstream-level evolution, circuit robustness, circuit transferability, evolvable hardware, evolutionary computation, field-programmable gate array evolution, intrinsic evolvable hardware, intrinsic analog evolvable hardware design

I. INTRODUCTION

EVOolvable hardware offers an innovative approach to circuit design, challenging traditional human-designed circuits under certain circumstances. This field stands to address computing efficiency and power consumption challenges inherent in conventional *digital* hardware, which often rely on standardized abstractions such as clock signals and discrete voltage levels. These abstractions, while easier for humans to comprehend (and granting certain guarantees of reliability), can lead to inefficiencies in power and computation. *Analog* computing, which does not necessitate clock synchronization and discrete voltage levels, has the power to enhance computing power and reduce energy consumption. However, designing effective analog circuits is complex, especially given the non-linear characteristics of semiconductors and environmental variations. Within the field of evolvable hardware, evolutionary algorithms can be

applied to reconfigurable hardware to evolve analog circuits intrinsically. The evaluation of the circuits embedded in the hardware can leverage non-linearities to enhance rather than degrade their performance. This approach, exemplified by Adrian Thompson's work in evolving a tone discriminator with minimal logic gates [1], demonstrates the potential of evolvable hardware to perform complex tasks efficiently. This result, along with several other experiments, has only recently been reproduced on modern hardware [2], after a hiatus due to several factors [3].

This recently resurrected research area is one subdomain of evolvable hardware, which is itself a broad field, divisible into several categories of practice - intrinsic vs extrinsic, analog vs digital, and adaptive hardware vs evolvable hardware design. As a whole, evolvable hardware aims to offer alternative approaches to the traditional, human-engineered designs of circuits. The toolkit we report in this paper is focused on in-

trinsic analog evolvable hardware design (EHD). Consistent with the EHD approach, in which one final circuit design is created during the evolutionary phase, we aim to evolve a final circuit that does not require changes during its operation. Due to this constraint, we must consider methods of improving circuits' transferability and robustness, which we explore in this paper. Given that analog circuits may be influenced by subtle environmental factors — unlike digital circuits which abstract them away — we chose to evolve circuits intrinsically, evaluating fitness from physical circuits, rather than extrinsically — i.e. via simulation — to ensure the best performance on actual hardware. This approach presents great opportunities to take advantage of signals and timescales that are below the level of abstraction in conventional digital hardware — allowing for evolution to produce more power-efficient circuits than could be designed by humans. However, this avenue also introduces the challenge of evolution relying too heavily on noise that is inconsistent between environments — resulting in the aforementioned focus on transferability and robustness.

One longstanding challenge for research in this field is the barrier-to-entry as existing efforts tend to require expensive hardware [4] and complex configuration to begin even basic experimentation. Our key contribution in this paper is lowering the barrier-to-entry substantially through the communication of the *Bitstream Evolution* platform as a well-tested and easy-to-use system. Our publicly-shared code repository can be found at github.com/evolvablehardware/BitstreamEvolution.

The system enables research in this subdomain of evolvable hardware by a wider range of people than ever possible before. There are many interesting research questions in the field and our inexpensive platform allows progress to be made in answering them. Our initial use of this technology platform has allowed us to explore the 1) robustness of different types of circuits, 2) methods to bridge the gaps between large timescales (an analog circuit, operating on the order of *nanoseconds* or less, producing an oscillating signal on the order of *kilohertz*), 3) algorithms to improve the transferability and robustness of circuits, as well as 4) explore what functionality can be evolved on modern hardware.

In the remaining sections of this paper, we will discuss in greater depth the goals (II) for this version of the framework, previous related work (III), as well as an overview (IV) of the *Bitstream Evolution* framework as a whole. In addition, we report on preliminary results (V) in evolving oscillators, and evaluations of the sensitivity and transferability of evolved circuits. Finally, we present a discussion (VI) the implications of the results as well as future directions (VII) of this research, some of which is under development and ongoing at this time.

II. GOALS

The main goal of this framework is to provide a widely accessible research and development platform. Greater community involvement has the potential to rapidly increase progress in the field. Our framework, *Bitstream Evolution*, is a well-tested, easy-to-use, and easily configurable software

framework for intrinsic analog FPGA evolution. Moreover, *Bitstream Evolution* is an open-source platform that utilizes relatively inexpensive and readily available third-party hardware components, making it an affordable option compared to most existing evolvable hardware research configurations (an iCE40 FPGA and Arduino Nano MCU have a combined cost of less than \$200, as of June 2025, via online suppliers such as [digikey.com](https://www.digikey.com) and store.arduino.cc). In addition to software used to control and measure signals from the FPGA, there are simple simulation modes available to verify the functionality of specific software modules such as fitness functions and evolution algorithms.

We aim to foster an open science community through several design choices. First, we have chosen to develop our software with an open-source license. Second, we use a publicly-shared repository of evolved bitstream populations to allow researchers to reuse preexisting evolved populations and contribute new ones.

In line with our goals of accessibility and open science, we have identified several open research areas; we report on our preliminary efforts in Section V. A key goal for the new era is exploring intrinsic evolution on modern hardware. We aim to identify best practices for evolutionary algorithms and evaluate the impact of various design decisions on noisy fitness functions [5].

We also want to answer questions about the characteristics of *generative* circuits which were evolved to utilize noise (they are not provided any explicit input signals). How easily are such circuits evolved? Are such evolved circuits robust in their behavior over time? In the same or different environment?

Similarly, there are open questions about the relative challenge and reliability of circuits that exhibit dynamics at different timescales. Are circuit behaviors more easily evolved at certain frequencies? What causes any observed differences in behavior?

Valuable questions remain about evolving robust circuits. What techniques can adjust algorithms, fitness functions, and hardware to enhance circuit consistency? Does varying the operational envelope during evolution improve self-consistency? Can multiple evaluations of a single circuit within a fitness function improve self-consistency?

Beyond robustness, we aim to investigate circuit transferability. Specifically, how do circuits evolved on one FPGA perform on others? We seek methods to improve transferability, such as parallel evaluation on multiple FPGAs or alternating evolution on different FPGAs (see Section V).

In summary, we hope that the use of this highly accessible system will help address questions in areas of 1) best practices for evolutionary algorithms on modern hardware, 2) characterizing properties of circuits evolved for generative tasks utilizing noise, 3) impacts of timescale on the evolution and behavior of circuits, 4) identifying effective methods for increasing robustness and 5) transferability for circuits. We hope to see the growth of a research community with

standards of open source, open science, and community collaboration.

III. RELATED WORK

In this section, we provide a summary of past related research, and recent developments in evolvable hardware related to the goals of this project. We look at an analysis of the challenges the field faces as a whole, Thompson's original research, the resurrection of FPGA-intrinsic evolvable hardware, the first successful tone discriminator evolved on modern hardware, and evolution performed on field programmable transistor arrays (FPTAs) to recognize spoken commands.

Haddow and Tyrrell examine the past, present, and future of evolvable hardware in [6]. They examine previous successes and discuss the challenges facing the field. One main challenge they identify is the need to publish, as new fields can struggle to publish meaningful findings that demonstrate their applicability (comparing the field to neural networks and the decade-long slowdown that field experienced). We hope this framework will serve as a means for making further discoveries worth publishing. The second problem they identify is scalability - the ability to solve increasingly complex tasks without providing further resources such as components or time. While the field has produced some promising and interesting results for relatively simple problems, it is important to be able to use EHW techniques for larger problems within reasonable constraints. While *Bitstream Evolution* does not directly address this issue, we hope that increasing accessibility and interest in the field will lead to further work to tackle this key issue. The final issues that the author identify are measurements (specifically what the community should be using to measure the success of a circuit - not area or generations, but reliability) and difficulty interacting with minute defects in different circuit boards. These both relate to the ideas of robustness and transferability that have begun to be explored in this paper. Working towards these two goals has been a main objective of the *Bitstream Evolution* framework thus far, and we hope that others can push the boundaries further.

Cancare et. al. developed their own system [7] for evolving hardware on Virtex4-based systems, similarly to the role *Bitstream Evolution* is intended to fill for the iCE40. This EHW platform utilizes a unique feature on these types of FPGAs - two-dimensional reconfigurability, allowing for subsets of the hardware to be reconfigured dynamically. This feature allowed them to quickly perform multi-grained evolution, in which a mix of higher-level and fine-grained evolutionary techniques are used. While they demonstrated the success of their system on a variety of tasks, the most complex and interesting task was a circuit that could solve the problem of balancing an inverted pendulum (given the cart position and current angle). They accomplished this by starting with a higher-level evolutionary process until fitness surpassed a prespecified threshold, then switching to a fine-grained (direct bitstream manipulation) process to refine the solution. The *Bitstream Evolution* framework currently only supports

direct bitstream manipulation, and is able to succeed in simple tasks using this method, but a direction of future work is in devising a higher-level intermediate representation (as done by Concave et. al.) to more quickly converge on more optimal solutions for complex problems.

In 1998, Thompson tested evolving robust circuits by running the evolutionary algorithm on multiple FPGAs in different environmental conditions, a configuration he dubbed the "Evolvatron." Most circuits evolved on a single FPGA in a constant environment tend to only be functional within a particular testing environment; small changes can have dramatic impacts on a circuit's performance. While he made progress towards evolving robust circuits, Thompson was not successful in evolving circuits that worked consistently across multiple boards; instead, evolution seemed to trade off success on one board for more success on others. [1]

In [3], after a slowdown for the field of intrinsic analog FPGA evolvable hardware design, Whitley et. al. were able to evolve simple oscillators on an iCE40 FPGA, demonstrating the potential for the field to continue being studied using modern hardware. However, that initial effort was unsuccessful in replicating Thompson's original work in creating a consistent tone discriminator circuit on the iCE40.

While the *Bitstream Evolution* system underwent development, Fritzsche et. al. [2] successfully evolved a tone discriminator on an iCE40 FPGA. A contributing factor to their success came from an improvement in the speed of reconfiguration of the FPGA, with each reconfiguration reportedly completing up to 130 times faster than the *Bitstream Evolution* toolkit currently provides. This is accomplished using both an FPGA that allows for direct configuration (bypassing flash memory), and a compact bitstream method instead of the IceStorm tools. However, it is important to note that the fitness evaluation time remains unchanged — and, because fitness is by definition measured over long periods of time (on the order of seconds), this presents a substantial bottleneck on any platform. They found that their tone discriminator, like Thompson's, responded to temperature changes; however, it showed greater dependence on its position within the board, experiencing a fitness drop by orders of magnitude when relocated to different areas of the FPGA. It is currently unknown why their circuit was so much more location-dependent than Thompson's original work [8].

Recently, Garvie et. al. [4] extrinsically evolved robot controllers on an FPTA for visual tasks, using a low-quality camera to add noise as an intentional further challenge for evolution. Additionally, they evolved circuits for recognizing two simple spoken commands - a function that, as will be discussed later (Section VII), as a future direction of analog FPGA-intrinsic EHD. Their existing research paves the way for high-efficiency, low-cost computation. Reproducing a similar result on cheaper hardware (FPTAs tend to be significantly more expensive than iCE40 devices) could make this form of computation available to a wide pool of researchers. In the following section, we outline our technology platform which we hope will make this a reality.

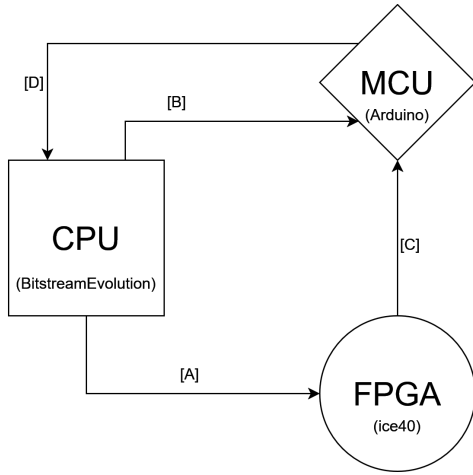


FIGURE 1. System Diagram

IV. PLATFORM FRAMEWORK

Bitstream Evolution is our technology platform which includes both the hardware and software for evolving circuits. Figure 1 provides a simple diagram of the overall system, and the paths of communication between different components. The hardware includes an iCE40 FPGA for configuring and evaluating the circuits (FPGA), and an Arduino Nano microcontroller (MCU) for communicating between the software running on the host CPU and the FPGA, and of course, the host which runs the BitstreamEvolution tool (CPU). Connection [A] in Figure 1 specifies the USB link for the CPU to upload bitstreams to the FPGA. Connection [B] specifies another USB link from the CPU used to send commands over serial to the Arduino, which tell it to begin recording a measurement and what type of measurement to record. Connection [C] shows a wired connection (single-wire) between the FPGA and the Arduino, through which the Arduino reads either digital or analog output from the FPGA. Finally, connection [D] represents the same physical USB link at connection [B], but logically is used to send the collected data from the Arduino back to the host CPU to be interpreted. The software, written in Python, allows a user to specify various parameters for an evolutionary algorithm, as well as circuit fitness criteria. The system utilizes a low-code approach to specifying these parameters through the use of configuration files, such that the existing code does not need to be modified in order to run new experiments.

During evolution, the CPU generates and maintains a population of individuals (bitstreams). The CPU will compile these bitstreams and upload circuits to the FPGA, asking the microcontroller for readings. The FPGA readings are transmitted via direct wired connections to the microcontroller, which reads them and returns them to the host CPU. The host CPU will use the recorded data to determine an appropriate fitness score, run the evolutionary process according to configured parameters, and create the next generation for evaluation and modification.

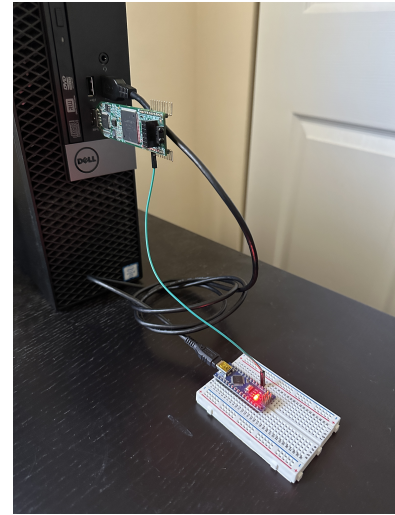


FIGURE 2. Hardware Setup for the System, including a desktop, iCE40 FPGA, and Arduino Nano connected via USB cables and a wire.

A. HARDWARE

Along with a host CPU, our system works with Lattice iCE40 HX1K FPGAs and Arduino Nanos which are shown in Figure 2 (both of which can be purchased online on sites such as digikey.com or store.arduino.cc). We have used the iCEStick Evaluation Kit development board, but other platforms providing similar access to an iCE40 HX1K device should be compatible. Both the iCE40 HX1K and Arduino Nano were chosen for their low cost and common use in open-source tools. With Arduino boards, we can easily read and record analog or digital signals produced by the FPGA. Through the use of serial communication, these signals can be sent to the host CPU for analysis.

1) Lattice iCE40 HX1K FPGAs

For the FPGA, Project IceStorm [9] provides an open-source toolkit for analyzing and modifying the bitstreams of circuits. These tools allow direct modification of the bitstreams of FPGAs. This opens the door for exploitation of the hardware (taking advantage of non-linear effects), with a set of defined components.

In our experiments, we only modify the bits of the logic tiles. These bits control how the tiles are routed together and what function each tile performs. We restricted our experiments to only the modifications of bits in a contiguous block of 96 logic tiles. This is both due to the immense size of the search space (160 total logic tiles, each with 864 bits, for a total of 138,240 bits), and to get as close as possible to Thompson's original experiments, which utilized 100 logic tiles [?]. The rows and columns in each tile that can be modified throughout evolution are controlled by a user-defined configuration. Additionally, the system supports multiple I/O configurations for the FPGA.

Many settings were fixed across all the experiments reported on in this paper, and we provide example configuration files specifying these settings in the github code repository.

Examples settings include modifiable regions, the baud rate (115200) for the Arduino Nano, and the I/O pins. Finally, Project IceStorm provides support for converting the bitstream files (stored as ASCII characters for easy modification) into binary files for compiling and uploading.

2) Increasing iCE40 FPGA Programming Speed

Although not required to use our platform, there is a documented means of increasing the programming speed of the FPGA itself. Project IceStorm documents the process for this hardware modification, which involves jumping the S RAM chip. This allows for direct writing to the FPGA and using the `-S` argument in the `icestorm` tool. Writing directly to the FPGA means that the data is not stored in memory and thus will not be saved after a power cycle. Fortunately, this is not a problem for the system since we always reprogram the chip before we cycle power. The exact speed-up may vary based on local configuration, but in one of our test configurations, the time to program was reduced from about 9 seconds to 1 second. Instructions to make this modification can also be found on the public *Bitstream Evolution* hardware repository.

3) Disabling iCE40 FPGA Clock

The clock on the FPGA is disabled in each of the experiments we report on. The purpose of this is to explore the capacity for evolution to discover circuits capable of generating signals at different timescales without relying on a clock. In addition to no clock signal, no input signal is directly provided to the FPGA as we currently focus on generative tasks. It is worth noting that disabling the clock allows for, and encourages, the possibility of the FPGAs picking up on an external signal in the environment and using that throughout evolution.

4) Arduino Nano

The system uses Arduino Nanos as our primary way of receiving data from the FPGA. Nanos were chosen because of their low cost (less than \$25), expansive documentation, and ability to read both analog and digital signals without any additional hardware. During evolution, the CPU uses the serial interface to tell the Arduino when and what type of data to record. The Nano uses either its analog-to-digital converter (ADC) to record a circuit's waveform or digital interrupt pins to record logical transitions. When recording the analog waveform, we take 500 samples, one every 10 microseconds, for a total sampling time of 5 milliseconds. When recording the number of pulses, we use one of the Arduino's interrupt pins to record the number of times the FPGA output transitions from low to high over an interval of 1 second. This data is then sent back to the CPU, using the serial interface, for further processing. We plan to extend support to include additional microcontrollers in the future.

5) Aliasing Issues with Arduino Nano

The primary limitation of the current approach lies in the overall accuracy constraints of the Arduino Nano. In this

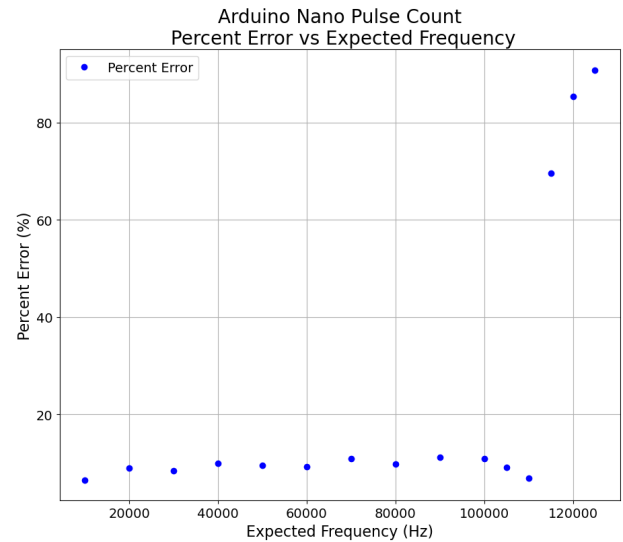


FIGURE 3. Arduino Nano Interrupt Accuracy Graph. Showing the percentage error for the readings when a particular ground truth signal is provided as an input to be read.

current version, since we are not filtering out signals that surpass the reading precision of the interrupt pin, we encounter the issue of signal aliasing. Specifically, certain signals may surpass our reading capability, and when attempting to read such signals the resulting data may be inaccurate.

As seen from Figure 3 anything over about 110 kHz is not accurate. Despite this, we can evolve an oscillatory circuit that will produce a certain interrupt report on an Arduino. However, in principle, this report could be caused by several different frequencies. For the oscillatory experiments we report on, although we targeted frequencies in the more accurate range of Arduino readings, there may be differences in the reported frequency measured and the underlying waveform.

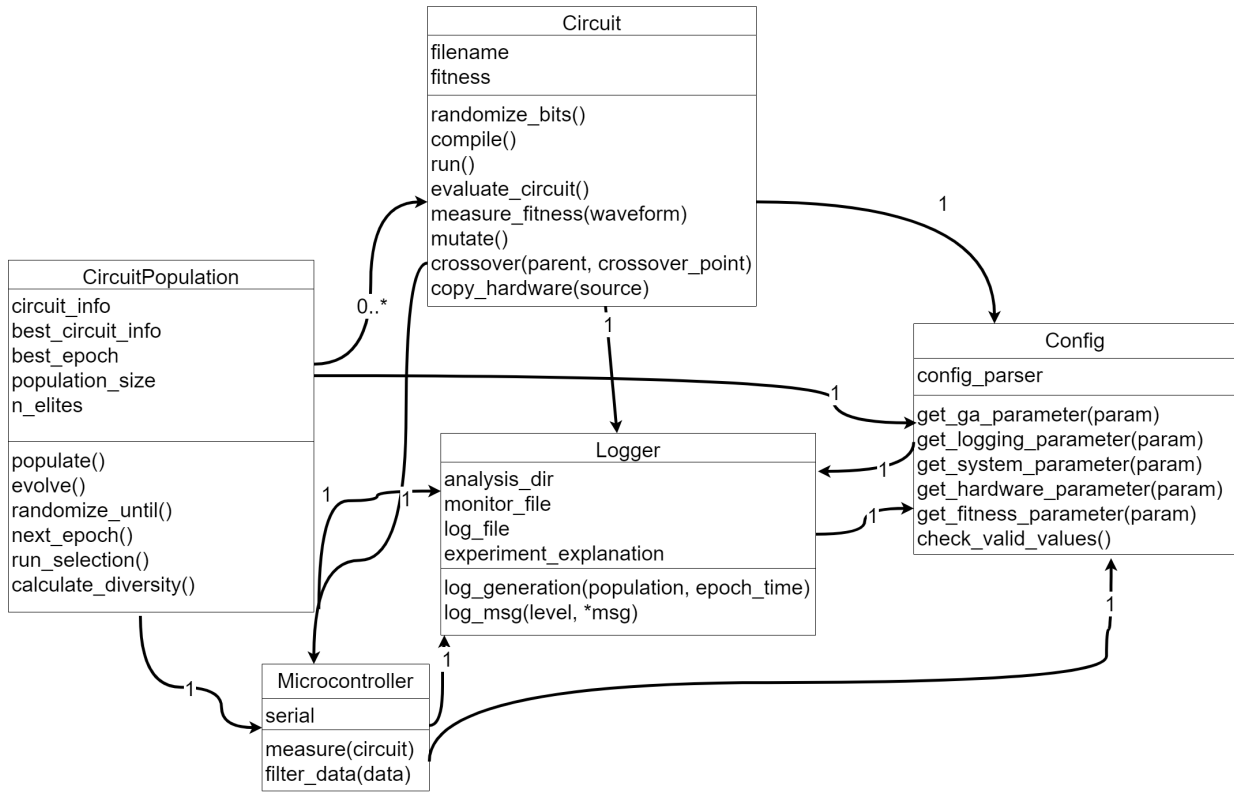
To combat these issues, we have considered adding a low pass filter between the Arduino Nano and the FPGA (Figure 1 [C]) to limit the frequencies being passed into the Arduino as only those within a sufficiently accurate measuring range as shown in Figure 3. One alternative approach would be to use devices with greater accuracy, such as ESP32 microcontrollers.

B. SOFTWARE

In this section, we focus on the “CPU” component of the overall system. We begin with an overview of the evolutionary algorithm to give context for specific features. Next, we discuss the population initialization settings and the different types of execution modes. Finally, we explain the methods of fitness evaluation and selection of successive generations.

1) Evolutionary Algorithm

The evolutionary process starts with initializing the population, which may start with randomization (see Initialization Modes). Figure 4 provides a high-level design overview in the form of a class diagram. The population and evolutionary

FIGURE 4. Class Diagram of the *Bitstream Evolution* system

process is managed by the *CircuitPopulation* class. After population initialization, during each generation, the *CircuitPopulation* class checks if evolution should terminate. It then evaluates and sorts the *Circuits*. Each *Circuit* instance is responsible for evaluating itself (see section IV-B4 and managing its own hardware file (performing mutations, crossover, etc)). The microcontroller itself is encapsulated in the *Microcontroller* class, of which the program has a single instance that all other components share. The *Microcontroller* class is used to gather measurements for calculating fitness, which the *Circuit* does once it gathers its readings. Upon gathering fitness values, the *CircuitPopulation* performs selection (see section IV-B9), followed by mutation and crossover. At this stage, it injects random circuits if specified to do so. Finally, all plot data is updated, and the next generation begins.

TABLE 1. Initialization Modes

Mode Name	Simple Description
RANDOM	Completely random bitstream
CLONE_SEED	Clone the specified seed circuit to fill the entire population
CLONE_SEED_MUTATE	Clone the specified seed circuit to fill the entire population, but also mutate it
EXISTING_POPULATION	Clone an existing population from a specified directory to fill the entire population

2) Initialization Modes

Given the slow evaluation time of circuits, it is important to try to start with an initial population with its best chance of success. Based on different goals, different initialization methods may be useful, and we provide and explain several of them in this section. Table 1 provides a high-level description for each of these methods.

The *fully random mode* (RANDOM) will randomize all (modifiable) bits in each circuit of the starting population, offering the highest initial diversity. This may make sense to use when trying to evolve a circuit for the first time on a new task or new hardware.

The *clone from seed* (CLONE_SEED) option will use a seed individual circuit and make copies of it to create the entire initial population. This causes experiments to start with very low diversity but can be used to improve upon a known high-fitness solution. This may be useful when trying to continue an evolutionary run from either the same hardware or when measuring or testing transferability (see section V-B).

A similar variant is *clone from seed and mutate* (CLONE_SEED_MUTATE), which will clone a seed individual but stochastically mutates each resulting circuit, raising the initial diversity while also building off of an existing solution.

Both clone options can also be used to progressively guide evolution. For example, an experiment can be performed with one fitness function, and then the best individual can be taken

and used to seed another experiment. This gives the initial population of this second experiment a place to start, without having to evolve a challenging task from scratch (for example, running a variance-maximization experiment and using the resulting best circuit as a seed for an oscillator experiment)

The *existing population* (EXISTING_POPULATION) option allows users to specify a directory that contains sub-directories of circuits that will be cloned and used as the initial population. This allows evolution to run on a previously saved population. This option may also be useful when exploring the transferability of circuits from one board to another.

The system also has randomization modes which aid in initialization, by randomizing circuits until they meet some criteria. This option can be configured to completely randomize circuits until they reach a certain goal, or mutate circuits, allowing the initialization setting to set the starting population. For example, one could start with an existing population and proceed to mutate circuits until reaching some criterion. Randomization can also be disabled to use the initialization mode to determine the starting population.

3) Execution Modes

TABLE 2. Execution modes

Mode Name	Simple Description
INTRINSIC	Evaluates fitness on the FPGA
BITSTREAM_SIM	Compiles bitstream and sums the bytes in the file
WAVEFORM_SIM	Generates sine waves that can be treated as an input signal

Bitstream Evolution features different modes of bitstream evaluation, each modifying the behavior of the Circuit class, each of which is provided in Table 2. In the *intrinsic* (INTRINSIC) mode, circuits are evolved and uploaded to the FPGA - this is the only intrinsic evolution mode. The other modes were introduced as methods of verifying and testing other parts of the system.

The first true simulation mode, *bitstream simulation* (BITSTREAM_SIM) uses the files that are uploaded to the FPGA, but arbitrarily calculates fitness based purely on their bitstream (e.g. counting the number of 1s), rather than uploading them, for faster evaluation. This mode is useful for verifying that mutation and crossover are not corrupting the circuit files, and making sure circuits can be compiled properly.

The other simulated mode, *waveform simulation* (WAVEFORM_SIM) does not use circuit files at all, but rather it's own short, contained arbitrary bitstream. For this reason, every circuit in this mode will start with a default (all 0's) bitstream, unless the RANDOM initialization mode is selected (see Initialization Modes) — this is because the waveform bitstream is only 100 bits, and using hardware files to seed the initial population is not applicable. In this simulation mode, circuits will generate waveforms weakly similar to the noisy signals output by the analog pins on the FPGA. This mode is useful for testing selection methods and the evolutionary loop, as it allows rapid simulation without any FPGA interaction.

4) Fitness Evaluation

Currently, there are three different fitness functions used to evaluate a circuit while in the fully intrinsic mode: variance maximization, sensitive pulse count, and tolerant pulse count. Table 3 provides a simple description of each and the equations are provided below. The variance and pulse count fitness functions are not comparable. We chose to use different types of functions to highlight the system's ability to evolve circuits suited for a wide range of tasks.

TABLE 3. Fitness Functions

Function Name	Simple Description
VARIANCE	Evaluates the "noisiness" of a waveform
SENSITIVE_PULSE_COUNT	Evaluates the closeness of the recorded pulse count to the desired frequency, with fitness decaying rapidly as distance from the target frequency increases
TOLERANT_PULSE_COUNT	Evaluates the closeness of the recorded pulse count to the desired frequency, with fitness decaying less rapidly as distance from the target frequency increases

$$\Psi = \frac{\sum_i^N (|x_i - x_{i+1}|)}{N} \quad (1)$$

$$\Psi = \begin{cases} \frac{1}{|f-n|} & \text{if } n > 0 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

$$\Psi = \begin{cases} \exp(-0.5(\frac{f-n}{0.025*f})^2) & \text{if } n > 0 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

The first fitness function, *variance-maximization* (VARIANCE, equation (1)) focuses on maximizing the "noisiness" of the waveform. It takes the sum of absolute differences between consecutive points of the waveform (x_i, x_{i+1}) read in at discrete time steps, and divides that by the number of voltage readings (N). This means the measure of "noisiness" is the average difference between voltage readings on consecutive time steps. Since the circuit's waveform may vary each time it is measured, this fitness function is stochastic. However, as discussed in section V-C, the fitness tends to stay relatively consistent across trials in the same environment. For each evaluation, we take 500 samples delayed by 10 μ s each, for a total sampling window length of 5 ms.

To evolve oscillatory circuits that produce rising-edge pulses at a given frequency, we have developed and utilized two different fitness functions (sensitive and tolerant pulse count). We have also made adjustments to them to improve the quality and rate of evolved circuits. Each fitness function attempts to match the number of times (n) the waveform rises above a threshold voltage over a span of a second with a desired frequency (f), however, the sensitive function is more punishing towards measurements away from the target

frequency, as discussed later in this section. We count pulses using the Arduino's digital interrupt pin over a sampling window of one second. Thus, the pulses counted is the frequency of the circuit.

5) Addressing Hardware Timeout Issues with Software

One technical difficulty we discovered is that occasionally when evaluating a circuit, the Arduino receiving the signal from the FPGA would timeout, meaning that no result was returned to the CPU. This does not indicate that zero pulses were recorded, but rather that the Arduino never delivered a response. Our working hypothesis is that this may happen when a circuit produces so many rising edges that it continually interrupts the Arduino, thus preventing it from executing its return result commands. Although this might be addressed by changes in hardware (see section IV-A5), we elected to address this issue in software by recording such non-responsive behaviors as producing -1 pulses for the purpose of computing fitness (to distinguish from a true result of 0 pulses). If we do encounter a timeout, we attempt to get the data five additional times before moving on and assigning a pulse reading of -1 .

6) Encouraging Non-Zero Pulse Counts

A second issue we discovered was that when targeting certain frequencies, evolution would converge on circuits that produced zero pulses. Upon further investigation, we recognized that evolution would discover circuits with non-zero pulse counts, but in many cases these circuits would produce a frequency much higher than that. Under these circumstances, when the conventional fitness function ([1], [3]) was targeting a relatively low frequency (under 20 kHz), a circuit producing *zero pulses* would receive a *higher* fitness score than a circuit producing *a large number of pulses*.

Further compounding this issue is that we have found it very difficult to produce non-zero pulse counting circuits when randomly generating circuits. To address this we normally initialize the population with one or more seed circuits that have demonstrated a positive number of pulses. Although genetic drift *might* eventually arrive at a non-zero pulse counting behavior, this is not necessarily efficient, which is a concern given the exceedingly long evaluation time of each circuit. For this reason, we decided it would be best to reward circuits for *any* non-zero pulse counting behavior.

Based on these challenges, we created a fitness function (SENSITIVE_PULSE_COUNT, equation (2)). This was derived from past work [3] and was adjusted to incentivize selection for positive pulse counts (we temporarily added a $+1$ bonus to circuits that achieved any positive number of pulses, but subsequently determined this was unnecessary and simply assigning a fitness value of 0 to both circuits that caused a timeout issue or produced 0 pulses was sufficient; this would have had a marginal effect on the results and in cases where it was present, we recalculated fitness values for any data presented to compensate for the adjustment).

7) Increasing Tolerance to Frequency Variation

In the original fitness function from [3] and to a lesser extent this one, if a circuit was even one Hz off the desired frequency, its fitness would be dramatically reduced (hence, our name of *sensitive* pulse count function). This could lead to circuits with similar behavior having wildly different fitnesses. When using fitness proportionate selection (see IV-B9), this could lead to one circuit being over-selected and wiping out most of the existing population. Normally, this is not much of a concern, but because of the stochastic nature of the fitness function and inaccuracy of the Arduino there is a possibility of a single "lucky" but unreliable circuit was concerning. Given the occasional population crashes (sudden plummet of best and average fitness) that we had observed, we wanted to try to address this possible issue.

To counteract this, we added a second oscillator function (TOLERANT_PULSE_COUNT, equation (3)). This one uses exponential decay to help smooth the fitness distribution around the desired frequency. In a case where the desired frequency is 50k Hz, and a circuit produces 49,999 pulses, the sensitive pulse count fitness would be 1.5, while the tolerant pulse count fitness would be 1.99999968. Thus, we are attempting to account for the stochastic nature of our circuit evaluations by introducing more tolerance into fitness functions. However, in our experiments, we have not found a significant difference in the rate of evolution in the two functions, as discussed in section A-B. Identifying causes of population crashes and a more rigorous comparison of these two fitness functions remains an area for future investigation.

8) Multiple Samples and Passes

Both of the pulse count fitness functions support the ability to do multiple samples of each circuit or multiple passes through every generation. The system does support this for all our fitness functions, but the system currently assumes each of the samples have a fitness between 0 and 2, while the variance maximization fitness for a single sample can be above 300. We have not found a need to conduct circuit variance maximization experiments with multiple samples and passes. This is because circuits' variance maximization fitness measures are much more consistent than the pulse count fitness measures. Thus, there is a less need to encourage consistency when evolving circuits for variance maximization. When doing i samples, each circuit is tested i consecutive times. When doing j passes, every circuit in the generation is tested one after each other, and then this process is repeated j times. It is possible to do both multiple samples and passes in the same experiment. This results in a total of $i \times j$ measurements per circuit. The product of the measurements is used for fitness calculation (all measurements are in the range of 0 to 1). This product is taken before the additional fitness boost is given. In this case, the fitness boost ($+1$) is given if any of the samples of passes of a circuit record 1 or more pulses. We chose to use multiplicative aggregation (as compared to averaging or taking the minimum) to further encourage consistent circuits to be evolved. This also helps address the issue discussed in

the previous section of unreliable circuits being selected due to “lucky” evaluations.

Both multiple sampling ($i > 1$) and taking multiple passes ($j > 1$) are means of improving the consistency of individuals. The data for each pass is multiplied together (taking the lowest fitness from each sample batch), meaning multiple low-fitness readings are compounded, while one low-fitness reading will still have a large impact on an otherwise fit individual. Taking multiple samples is a simpler and faster way to record more measures than taking multiple passes. However, taking multiple passes will ensure a much larger time window in between each measure (or set of measures) being taken, giving more time for the environment to change. This aims to give circuits a harder time incorporating volatile environmental factors into their function, although taking i passes takes longer to run than taking i samples since circuits must be re-uploaded to the FPGA for each pass, while they only need to be uploaded for the first of the i samples.

9) Selection Methods

TABLE 4. Selection Methods

Selection Method	Simple Description
Fractional Elites Tournament	A set of circuits are defined as elite. Elite and non-elite circuits are paired such that non-elites get replaced
Fitness-Proportionate	Circuits' chances of being selected is proportionate to their fitness
Rank-Proportionate	Circuits' chances of being selected is proportionate to their fitness rank
Classic Tournament	Circuits are repeatedly paired up and evaluated, with the higher-fitness circuit surviving. Elites are excluded from the tournament.
Single-Elite Tournament	Same as classic tournament, but only one elite

To select circuits for each next generation, there are five selection methods, which are fractional elites tournament, fitness- and rank-proportionate, classic tournament, and single-elite tournament which are accompanied with a simple descriptions in Table 4. These selection methods were implemented primarily with the goal of keeping elitism high, since small changes to circuits can dramatically reduce their fitness. Thus, it is important not to mutate every circuit in the population, or else we risk losing the progress made during evolution. This is primarily accomplished by using tournaments in our selection method. Each tournament has a size of two, and only the worst-performing circuit is modified.

In addition to our selection methods, each run may use single-point crossover. Single-point crossover restricts the number of changes that are made to one circuit, while uniform or multi-point crossover would likely disrupt many of the FPGA's connections between logic tiles and functions and thus greatly reduce the circuit's fitness. Additionally, since the fitness evaluation is stochastic, a typically well-performing circuit might do poorly in one generation and be eliminated from the population. With high elitism, there is a greater likelihood this circuit will continue to be included (via duplicate

copies) in the population. With many duplicate or similar copies in a successive generation, a particular circuit that has been successful in the past and has a bad run can be removed during selection without a complete loss of the circuit, which lives on through its clones.

Fractional elites tournament selection (Alg. 1) starts by sectioning off a group of the fittest circuits (the elites) that will be kept the same in the next generation. For each non-elite circuit, a circuit is randomly selected from the pool of elites and the circuits' fitnesses are compared. Every non-elite will be involved in exactly one tournament, while an elite circuit could be involved in multiple tournaments. The algorithm outlines our process for how mutation and crossover occur on our circuits. This is a general way to ensure elitism remains high, while maintaining some diversity throughout the population. Every other selection method is based off this one, with modifications described below.

Algorithm 1 Fractional Elites Tournament Selection

```

1:  $P \leftarrow$  current population of circuits
2:  $N \leftarrow \emptyset$ 
3:  $E \leftarrow \{\text{the best } n \text{ circuits}\}$ 
4: for  $c$ , a non-elite circuit do
5:    $r \leftarrow$  a random choice from  $E$ 
6:   if  $\text{random}(0, 1) \leq \text{crossover probability}$  then
7:      $c \leftarrow$  crossover from  $r$  to  $c$ 
8:   else
9:      $c \leftarrow r$ 
10:  end if
11:   $N \leftarrow N \cup \{\text{mutate}(c)\}$ 
12: end for
13: return  $N$  as the new population

```

Fitness-proportionate selection functions similarly to fractional elitism; the only difference is in the probabilities of each elite circuit being selected for comparison. Here, the probability of a random elite being chosen for any given tournament is its fitness divided by the sum of all the elites' fitnesses - i.e. Roulette Wheel selection. Again, each non-elite circuit is involved in only one tournament.

In rank-proportionate selection, the probability of an elite circuit being selected is based on its rank in the population instead of fitness. This generally keeps more diversity in the population, since having one extremely fit circuit with fitness proportionate selection means most of the next generation will end up with bitstreams similar to that circuit.

In single-elite tournament selection, only the best circuit in each generation is selected to be an elite, but otherwise works the same as fractional-elite tournament selection. This selection method is used when diversity in the population is prioritized, as only one circuit is kept the same from one generation to the next.

For classic tournament selection, two circuits are randomly selected for the tournament at a time, without repetitions. The circuit with the higher fitness is treated as the elite. Using a tournament size of two (the only size the system

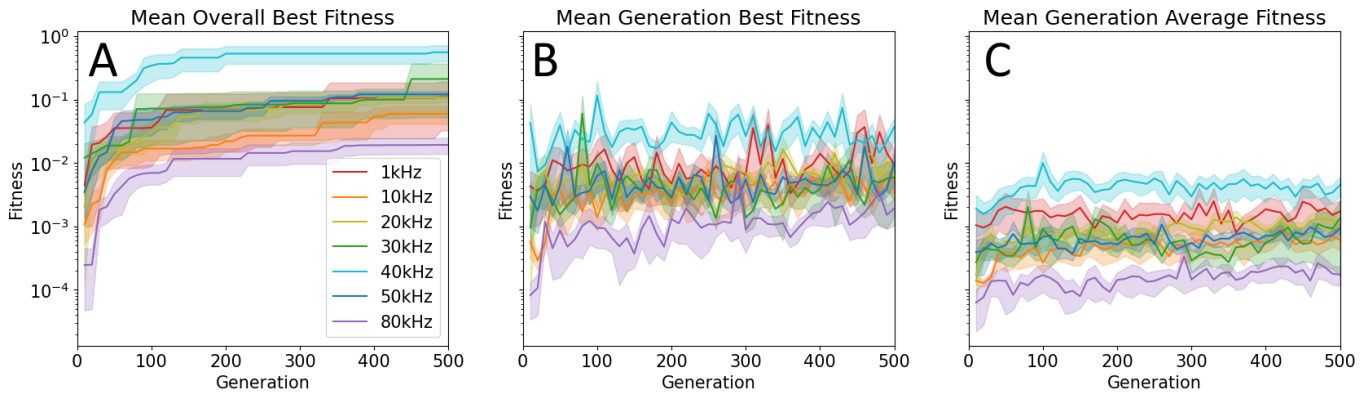


FIGURE 5. Overall Best, Generation Best, and Generation Average Fitness for different target frequencies, taken every 10 generations

currently supports), half the population is kept the same from generation to generation. This is most useful when we want to have a relatively stable population that minimally changes from one generation to the next. This method was inspired by the Microbial Genetic Algorithm [10].

V. ILLUSTRATIVE RESULTS

In this section, we report on experiments that can be easily run with the *Bitstream Evolution* platform and illustrate the insights that can be gained in doing so. These are not meant to be complete, comprehensive studies, but instead a demonstration of preliminary results that can be obtained using the Bitstream Evolution system. We first report on the results of running experiments with the target frequency fitness function, which aims to evolve oscillatory circuits matching rising edge signals to a specific frequency. Second, we report on the results of transferring and evolving circuits for both the target frequency and the variance maximization fitness functions. Finally, we report on the results of fitness sensitivity analysis in which we explore the variability of fitness of single circuits repeatedly evaluated in the same physical configuration to assess robustness.

A. PULSE COUNT TARGET COMPARISON

One of the core challenges of evolvable hardware from its outset is bridging the timescales of signals in the biological and electronic domains. Because of the advances in the manufacturing of physical components (e.g. wire and transistor sizes) of electronics in the last twenty years, results from Adrian Thompson's work required demonstration of feasibility on modern hardware, which was done in [3]. However, there remains an open question of evolution's ability to find circuits capable of operating at varying timescales.

To gain insight into the relationship between timescales and the evolvability of circuits, performed experiments comparing the evolution of (oscillatory) circuits targeting different frequencies. We hypothesized that generally higher frequencies would be easier to evolve than lower frequencies because the rate of propagation of electric signals is so high. During initial experimentation, we often observed randomly

generated circuits starting with a frequency of around 40 kHz. Because of this, we decided to explore a range of target frequencies above and below this rate, with experiments that varied only in the target frequency. Due to the time it takes to run these experiments, and our desire to test a range of target frequencies well above and well below 40kHz, we selected seven target frequencies that are a great distance from each other. For each of the seven target frequencies in Figure 5, we performed six experiments (on a mix of five different FPGAs), and averaged their results. The means across experiments are reported, specifically A) the best fitness reached by any circuit at any point during an experiment, B) the best fitness of the current generation, and C) the average fitness of the current generation appear in the figure.

The 40 kHz experiments consistently outperformed the others in each metric. The generation best (Figure 5.B) for the 40 kHz experiments was often on par with the overall best (Figure 5.A) for other frequencies, and the generation average (Figure 5.C) was generally on par with the generation best (Figure 5.B) for the other frequencies. In all metrics, the performances of different frequencies compared to each other are mostly consistent; for example, 80 kHz experiments performed the worst by every metric, and the 1 kHz, 20 kHz, and 50 kHz were often close together. This data suggests that some frequencies are easier to evolve than others. But, it's not purely based on proximity to a specific frequency, nor that higher frequencies are easier to evolve; the 30 kHz and 50 kHz experiments were similar to both the 1 kHz and 20 kHz experiments.

One might suspect that an 80 kHz oscillator would be "easy" to evolve if a 40 kHz is easily evolved given that it is exactly double that of 40 kHz. Such a solution might be possible if the same signal that is producing the 40 kHz signal is simply introduced into a parallel circuit that triggers the pulse twice as often. However, the 80 kHz experiments perform the worst out of all the targets, so the mathematical relationship between target frequencies may not be significant.

The progression of the best overall fitness of the various oscillators is very similar. It looks like the final overall best fitness is strongly correlated by the starting point. We suspect

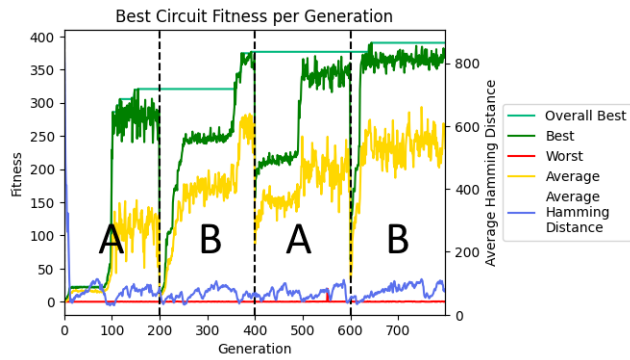


FIGURE 6. Fitness of circuits throughout an experiment exploring the transferability of a circuit evolved for the max variance fitness function. Evolution occurs while alternating between FPGA "A" and "B" every 200 generations, as depicted in the plot.

that the FPGAs are utilizing an environment-specific signal that makes 40 kHz an easy frequency to evolve for. Because these experiments have been performed on a variety of FPGAs in a variety of environments, we can rule out minute manufacturing differences and most environmental signals, as these are not likely to be common between all setups. The factors that could contribute would then be those that are common between all setups — such as the microcontrollers, which are all the same model. More work is needed to determine the exact cause of this phenomenon. Another direction of future work is comparing the structure of the produced circuits across the different experiments.

B. TRANSFERABILITY

We also ran experiments to explore how circuits can adapt when being transferred from one FPGA to another during evolution. To do this, we started from a random population of circuits, evolved them on one FPGA for 200 generations, switched to a different FPGA and evolved on that one for another 200 generations, and then continued to switch back and forth. These experiments used a population size of 50 and completed 800 total generations.

The *Bitstream Evolution* system allows the user to specify any transfer interval (TRANSFER_INTERVAL) to control how quickly evolution switches between two FPGAs. To accomplish this, two FPGAs need to be plugged into the CPU and connected to a single Arduino Nano. The system will automatically handle the switching process. Because of this, the circuits can all be evolved in a relatively constant environment. The temperature, humidity, and other features of the test site can fluctuate throughout the experiment, but by having the system automatically switch between FPGAs, we can help limit the number of changes that occur from one transfer interval to the next. By limiting the changes in the external environment, we can focus on how minute hardware differences between FPGAs affect the behavior of circuits.

1) Variance Maximization Circuit Transferability

The variance maximization experiment (Figure 6) shows how transferring between FPGAs can lead to the population escaping from local maxima in the fitness landscape. In the first interval, a maximum fitness of 320.742 was achieved during generation 155. The population was then evolved on a different FPGA for 200 generations, before switching back to the original FPGA. Here, it was then able to achieve a greater fitness of 376.79 during generation 392. Not much progress was made past this, possibly because in all intervals the population evolved to have its best circuits oscillate quickly between the highest and lowest possible voltage levels produced by the FPGA (Figure 7).

With every interval, the time it took to recover back to a high fitness decreased. We define recovery time as the number of generations to return to 95% of the maximum fitness of the previous interval. For the first transfer, it took 155 generations to return to 95% fitness, but the following interval took 110 generations, followed by only 22 generations. This shows that evolving circuits on multiple pieces of hardware can help make them more adaptable to changes in their environment, with circuits quickly recovering from being transferred. Additionally, this suggests that using a seed population from another experiment, even if the experiment was done on a different FPGA, can help get results faster than starting from a random population.

2) Oscillator Transferability

For testing the transferability of oscillators, we used our tolerant pulse count fitness functions (3), with a desired frequency of 80 kHz. Every circuit was sampled twice, and the worst measurement was used in fitness calculations. The purpose

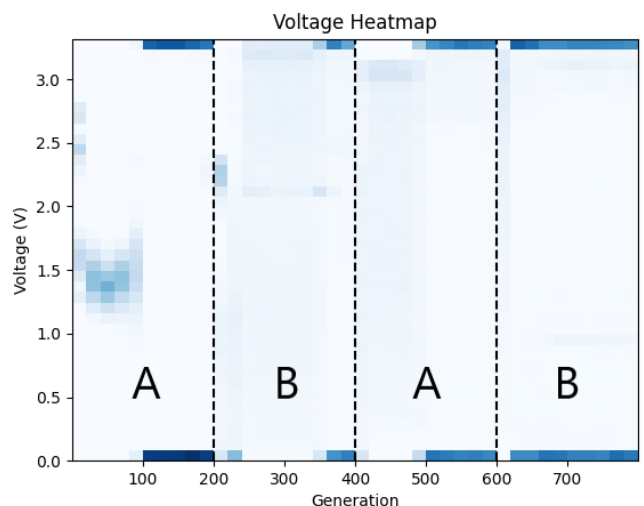


FIGURE 7. This heatmap (with darker colors indicating higher frequency) shows the voltages of the best circuits throughout an experiment exploring transferability of circuit evolved for variance maximization fitness. FPGA "A" and "B" are alternated every 200 generations, as depicted in the plot.

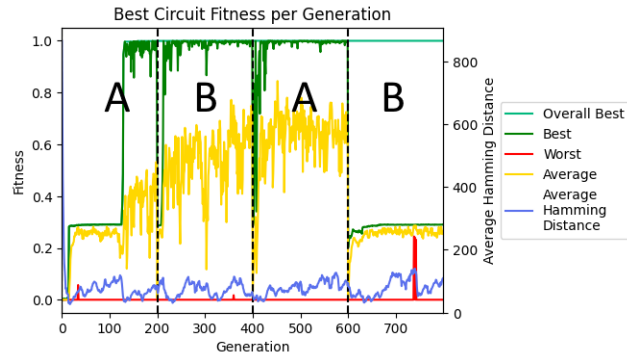


FIGURE 8. Fitness of circuits throughout an experiment exploring transferability of oscillators targeting an 80 kHz frequency. Evolution occurs while alternating between FPGA "A" and "B" every 200 generations, as depicted in the plot.

of using multiple samples was to evolve more consistent circuits. Our current recommended practice is to use fitness multiplication for multiple samples since it increases pressure on both consistency and quality.

This experiment (Figures 8 and 9) shows many of the same traits of transferability as the variance transferability experiment. For one, the time it took the population's best fitness to recover after a transfer point decreased throughout evolution. Additionally, the amount of the best fitness decreased by each transfer point also decreased for the first two transfer points. However, the last interval of generations saw significantly worse fitness than the previous intervals. Interestingly, the fitness was very similar to the fitness recorded in the first interval, despite being on a different FPGA. This means that transferability is not always effective.

C. FITNESS SENSITIVITY

In order to test how reliable our circuits are, we performed experiments taking single circuits and evaluating them repeatedly without making any changes to the circuit's bit-stream. This helps us quantify how robust our circuits are

when kept in a relatively stable environment. These circuits were tested in the same environment (same room and on the same hardware) they were evolved in, but differences in environmental factors, such as temperature and humidity, impacted the circuit's performance. Since the circuits were not tested in a specially controlled room environment, this gives us the opportunity to see how robust they are to these small environmental changes.

1) Pulse Count Sensitivity

For these experiments, we ran pulse count experiments (see Appendix A-B) with a variety of desired frequencies. These experiments were run on three different FPGAs, all in differing environments. The best circuit of the final generation of each experiment was tested repeatedly for 24 hours straight following the canonical experiment. We conducted 10 of these experiments, with desired frequencies ranging from 30 to 90 kHz, in order to determine if different target frequencies led to different levels of consistency in the evolved oscillators. The correlation between the mean frequency recorded throughout each experiment and the standard deviation of frequency recorded in the trials in each experiment is 0.28509. However, this could be due to the fact that a higher mean will naturally lead to a higher standard deviation than a lower mean. This is true even if most of the data points in each set are located within the same percentage of each mean.

Because we consider such a wide variety of mean recorded frequencies in these experiments, it makes more sense to use the coefficient of variation ($CV = \frac{\sigma}{\mu}$) to quantify dispersion in each experiment. Using the coefficient of variation allows us to compare the dispersion of recorded frequencies independently of the mean frequency recorded in each experiment. The correlation between the mean frequency recorded throughout each experiment and the coefficient of variation of the trials in each experiment is 0.02394. This data suggests that there is little to no correlation between the desired frequency and the consistency of an evolved oscillator.

We found that the distribution of the pulse counts stayed relatively consistent throughout the 24 hours in each experiment. The distribution of recorded pulses throughout one experiment is shown in Figure 10. The frequency produced by the circuit throughout the experiment was consistently around 50k Hz. However, there were still small periods of time where the recorded frequency of the circuit dropped, as shown in Figure 11. In a different experiment, we recorded the temperature and humidity during each trial (Figure 12). We found that a notable change in temperature and humidity coincided with a large drop in the recorded frequencies of the tested circuit. Even after the temperature and humidity returned to levels similar to that of the start of the experiment, the circuit fitness never rose again. This suggests that while temperature and humidity do influence a circuit's fitness, there are other factors influencing circuit behavior. Similarly, in Figure 11, we see a temporary drop in performance just before trial 8000 followed by a subsequent recovery, which could be caused by even something as simple as heating or

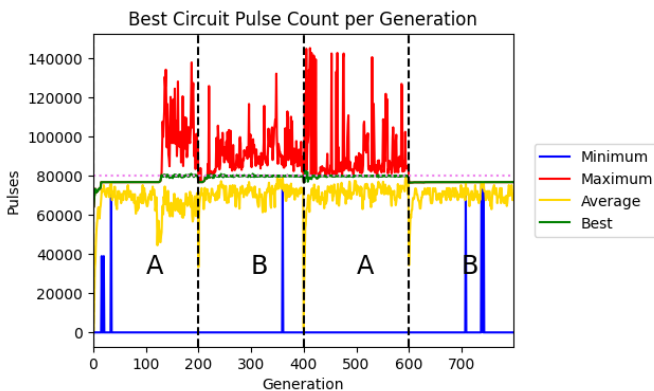


FIGURE 9. Produced frequencies of circuits throughout an experiment exploring transferability of oscillators targeting an 80 kHz frequency. Evolution occurs while alternating between FPGA "A" and "B" every 200 generations, as depicted in the plot.

cooling unit turning on. Despite this, it still seems that circuit fitness will remain consistent if the environmental conditions stay relatively stable.

2) Variance Maximization Sensitivity

To measure the robustness of circuits evolved for variance maximization, we ran a variance maximization experiment (see Appendix A-A), recorded the best circuit from every tenth generation, and tested each of these best circuits for three hours without additional changes. We chose to use a shorter sampling duration for these experiments because the fitness distribution stayed relative constant over time. An example of a variance maximization fitness distribution over 24 hours is shown in Figure 13. These tests were run consecutively to closely match the environment of each experiment.

We performed a similar analysis to that of the pulse count fitness sensitivity experiments. The correlation between the generation each circuit was pulled from and the standard deviation is 0.19169. The correlation between the generation each circuit was pulled from and the coefficient of variation is 0.004633. So, for variance maximization experiments, it seems that circuits are neither becoming more nor less stable throughout evolution. Additionally, fitness does not seem to correlate with consistency in behavior.

VI. DISCUSSION

Our results demonstrate that the *Bitstream Evolution* framework can be used to conduct experiments exploring the evolution and evaluation of circuit behavior. We have reported on the evolvability of circuits producing different target frequencies, explored the transferability of evolved circuits, and evaluated the robustness of evolved circuits.

Currently, the data suggests it is easier to evolve stable oscillators at certain frequencies. Generally, a desired frequency

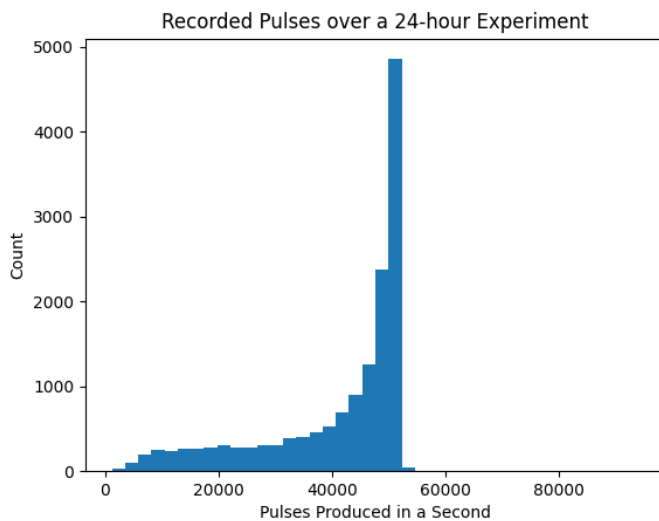


FIGURE 10. The distribution of recorded pulses over time during a pulse count fitness sensitivity experiment

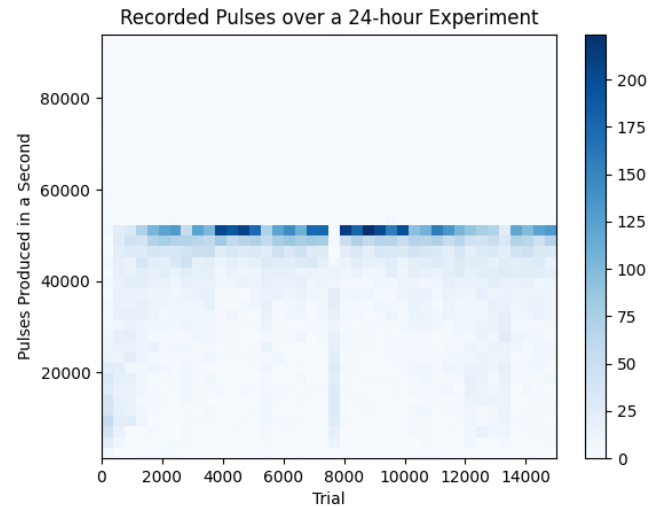


FIGURE 11. The distribution of recorded pulses over time during a pulse count fitness sensitivity experiment

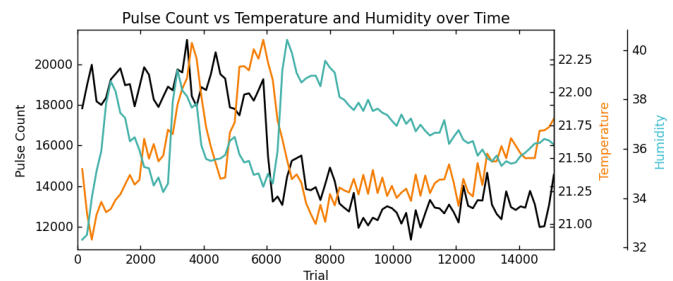


FIGURE 12. The recorded pulse count of a circuit vs the temperature of humidity of the local environment during a fitness sensitivity experiment

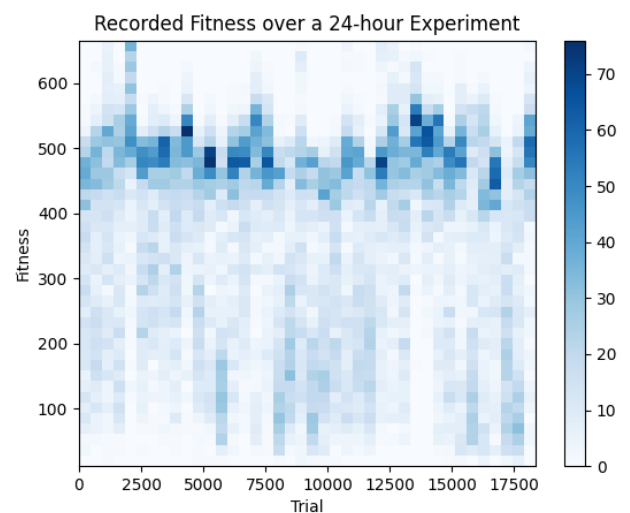


FIGURE 13. The distribution of recorded fitness over 24 hours during a variance maximization fitness sensitivity experiment

40 kHz seems like a particularly easy target for evolving oscillators. Every frequency tended to see similar progression, making rapid progress initially before plateauing by generation 200. From there, each type of oscillator would continue to make progress, but at a slower rate. The 40 kHz experiments made slower progress after generation 200, while some of the low-fitness oscillators were still making progress (such as the 10 kHz oscillators). The generation best and average fitnesses were sporadic for each type of oscillator. However, the 40 kHz oscillators tended to have a best generation fitness near the other oscillators' best overall fitness. There was similar behavior with the average vs best fitness. More work is needed to determine why a 40 kHz desired frequency tends to produce such stable oscillators.

While experiments in transferability are promising, there is room for improvement. When transferring a circuit from one board to another, we never saw the second board reach the same fitness as the first board right away, always requiring a few generations of evolution. From a practical standpoint, this is not feasible since EHD is intended to evolve a final form of a circuit; therefore, a future goal would be to evolve a circuit that could instantly perform on another board with no additional evolution. The first generation after transfer typically saw greater fitness than randomly-generated circuits would have accomplished, but in all cases, additional evolution on the new board was required to return to similar fitness levels. This evolution would take much less time to reach the fitness of the previous experiment, and in almost all cases was able to produce even better circuits. This suggests that using a previously evolved population of circuits could be used as a better starting population for further experiments, rather than a population of randomly generated circuits.

The variance maximization transferability experiments demonstrated the ability for transferability to escape local minimums and to decrease recovery time. Recovery time (generations to 95% of the previous interval's final fitness) decreased from 155 generations after the first transfer interval to just 22 generations after the final transfer interval. Ideally, one could evolve circuits with no recovery time and a seamless transition between intervals. While this has not been achieved, the decrease in the recovery time is promising. More work will need to be done to investigate whether properly general circuits can be evolved in this fashion.

The target frequency transferability experiment proved more challenging. Although the same pattern of decreased recovery time appeared from the first to the second transition, the third transition actually failed to recover with 200 generations. The population appeared to have converged to a very similar level of average fitness on *FPGA B* as it had originally done on *FPGA A* in the first 100 generations of evolution. Thus, it seems plausible that the components evolved in generation 200-399 on *FPGA B* aided the evolution in generations 400-599 on *FPGA A*. However, with the selection for characteristics of *FPGA A* or just genetic drift, it seems that the beneficial components for *FPGA B* may have been lost during generations 400-599. More thorough recording

of populations and subsequent experimentation and analysis could help verify or refute these ideas. These results parallel Thompson's "evolvatron" [1] both in the cyclic nature of evolution and circuit behavior, but also the challenge of producing a single circuit that operates well on multiple FPGAs.

Evolving on a single board in a single environment does not produce immediately transferable results (which is expected), so future work in evolving robust circuits may necessitate new methods to generate transferable circuits (discussed in section VII). However, as another step in making progress towards robust circuits, our experiments on measuring fitness sensitivity, or circuit self-consistency, have helped to illuminate the landscape of reliability for several example behaviors.

For sensitivity in variance maximization experiments, we have pulled circuits from various stages in evolution (different generations). While fitness improves over time (as expected), with the aforementioned correlation between the generation and coefficient of variation being 0.004633, fitness was equally stable for all circuits regardless of the generation the circuit was pulled from. Without specifically basing fitness on consistent behavior, evolution tends to find consistent circuits. It is likely just easier to evolve a consistently noisy circuit than an inconsistently noisy one. Because the time circuits are evaluated for is so short and randomness in timing (upload time, serial communication, CPU task scheduling, etc), the time a particular circuit gets evaluated is effectively random. We believe this is why circuits are equally stable no matter which generation they appear – an inconsistently noisy circuit would have a low chance of their period of "noisiness" coinciding with the measurement window. This bodes well for other applications, as we believe this property would extend to other types of fitness evaluation.

In the pulse count fitness sensitivity experiments, we examined the impact of target frequency and environmental factors on consistency. We found little to no correlation between target frequency and consistency, measured by the correlation between the mean frequency recorded and the coefficient of variation (which has a value of 0.02394). Additionally, when environmental factors are stable, pulse counts remain relatively stable as well. Significant changes in pulse counts coincided with significant changes in the environment's temperature or humidity, as we will discuss further below. Unfortunately, this means oscillators evolved in this fashion tend to fail in different environments, meaning in the future, further environmental pressure to evolve for robustness must be applied.

In the fitness sensitivity experiments, circuits showed relatively consistent fitness levels over time. Although the fitness functions used are stochastic, there was often not a significant variation in fitness for the same circuit. However, slight variations were present. Currently, we have minimal consideration made in the *Bitstream Evolution* tool to account for this stochastic nature. We do use high elitism in our selection methods and take multiple samples of each circuit every generation. With even these small considerations it seems possible to make consistent progress.

For the most part, a circuit's fitness seems to stay relatively consistent over time, but some environmental factors besides time, such as temperature and humidity, play a role in the behavior of evolved circuits. Beyond even temperature and humidity, we have evidence of at least one experiment (Figure 12) in which a single circuit evaluated over 24 hours had a drop in fitness corresponding to a sudden change in temperature and humidity. Interestingly, it never returned to the same fitness level again, even after the temperature and humidity returned to the previous measures. Thus, more work is needed to determine what other factors impact circuit behavior. Additionally, the fitness sensitivity of circuits seems to be independent of their average fitness, suggesting at present, evolution neither encourages nor discourages stability.

VII. FUTURE WORK

The FPGA-intrinsic analog domain of the field of evolvable hardware is still ripe for further development. In this section, we provide our vision for some short and long-term goals for the field. We start with goals to expand the *Bitstream Evolution* toolkit and then discuss some of the theoretical and empirical work that needs to be done to re-establish the field more rigorously. Finally, we outline several fields that we see this work having an impact on and articulate how that may happen in the future.

Most immediately, we plan to enhance this toolkit through the development and implementation of an open-source carrier board. This carrier board will support using a Raspberry Pi as the CPU, which will run the *Bitstream Evolution* software. The carrier board will support headers for easy connections to one or more FPGA development boards. The board will also be designed to allow modifying the PID of the USB adapter on the development boards. This will greatly simplify addressing the FPGA boards individually, enabling future experiments exploring transferability and evolution on multiple FPGAs simultaneously. Documentation for this ongoing work can be found on the *Bitstream Evolution* hardware repository.

With this new approach utilizing the carrier board, we plan to allow direct connections to the Raspberry Pi to transmit measurement data. The integration of functionality with the Raspberry Pi itself streamlines the design by eliminating the need for the Arduino Nano, which simplifies the overall setup and facilitates future scalability. Furthermore, this platform enables expandability for gathering additional measurements such as environmental temperature and humidity. The carrier board also features accessible headers for system inputs, which allows for complex experiments and easier external verification through oscilloscopes of the resulting circuit. We hope that this will aid in our ongoing efforts to reproduce the results of Thompson's tone discriminator, similar to that done by [11].

There are some broader empirical questions we likewise see on the near horizon for this field. We seek to understand the "ease" of evolving oscillatory circuits with an approximately 40 kHz frequency. Is there a specific source of noise

we can identify that allows this? A related but different question we are exploring is, how easily can one evolve a circuit that can function as a radio? Along this line of inquiry, one could ask what are the limitations and opportunities that such circuits could introduce? There are also still broader questions about the potential and limitations of robustness and transferability of evolved circuits. Many possible methods for defining fitness functions and population operators to enhance robustness and transferability are ready to be explored out-of-the-box with *Bitstream Evolution*. We are eager to see what we can discover as we explore ourselves or perhaps learn from others in the community. For instance, systematic exploration of the impact of passes and samples could offer insights into the effectiveness of selection pressure on self-consistency.

As mentioned in the related work (III), Thompson's work on the "evolvatron" seemed to indicate challenges for getting a single circuit to work on multiple FPGAs, but this was an early, preliminary work. With a renewed interest in the field, more work can be done to either overcome the challenges or provide insight into the nature of the limitations for circuit robustness and transferability. Relating to this are the opportunities afforded by the field of quality diversity [12], which had not yet been formed at the inception of evolvable hardware. There is an opportunity to explore the evolution of high-performing, but behaviorally-diverse, populations of circuits. We hypothesize that the diversity and quality of solutions produced should increase the odds of providing better transfer across different physical devices. However, even if circuits do not immediately transfer high-quality behavior, we believe that having a larger pool of diverse solutions greatly increases the odds for evolution to find a "foothold" to be used to initialize a new population to be evolved on new hardware.

While the work done to date has been performed at the level of bitstream manipulations (i.e. pseudorandom bit flips), there is an opportunity to devise intermediate representations, either through direct or indirect encodings of the genome for defining circuits. Incorporating these into the *Bitstream Evolution* is a promising direction, and our work may build on prior work in this area [13]–[15]. Although evolution has found ways to take advantage of disconnected components that still serve a functional purpose in evolved circuits, it would be interesting nonetheless, to aim for explicit mutations resulting in additions or deletions of connected components. Taking such an approach has the potential to both rapidly increase the rate of evolution and result in less time exploring circuits that might not have any functional impact on existing circuits. It also might result in the evolution of circuits that are more reliable or transferable given a more strict set of constraints. Although not yet reported, preliminary efforts to investigate the minimal number of logic blocks required to evolve circuits for some of the basic circuit behaviors reported in this paper are currently underway. Such efforts focus on generating the minimally sufficient qualitative behavior of an output waveform by constraining the

number of resources available throughout the evolutionary process, such as an analog pulse oscillator evolved with fewer than 25 CLBs or targeting the maximum signal variance for a circuit with 10 CLBs. Further investigation of these resource deprived circuits, as well as exploration of evolution using high-order representations of circuit genomes, is likely to reveal the deeper potential and challenges in the field.

As progress is made in these different areas it may open up new avenues in other fields. One area of work is in computer security. If it were possible to manipulate bits or read signals in disconnected components, it would open a wide range of possible exploits. For signal processing on embedded devices such as satellites, the need for efficiency is paramount, and often there may be limitations for power and physical space. For such conditions, evolvable hardware stands to push the bounds of what is feasible with creative uses of electronic components. Additionally, usage of these algorithms allows for circuits to be fine-tuned for specific Digital Signal Processing (DSP) applications or use cases. These specialized circuits can have performance benefits over the general-purpose tools and algorithms commonly used in DSP applications, in terms of performance, power usage, efficiency, physical space, etc.

Beyond these areas, with the ongoing AI revolution, the need for more efficient and neuromorphic approaches to hardware is as promising as ever. The long-standing von Neumann architecture is facing increasing strain, and increasingly specialized or reconfigurable hardware is being developed and deployed. The intrinsic analog evolvable hardware research we seek to advance with this toolkit is being applied to an FPGA designed for digital computation, but can help to answer questions about the opportunities and limitations of evolvable hardware more broadly. For instance, recent work in hardware accelerated neuroevolution [16] has shown that utilizing reconfigurable hardware such as multi-processor system on chip (MPSoC) based FPGAs and methods of intrinsic evolution drastically increases circuit evaluation and the rate of evolution for Continuous-Time Recurrent Neural Network models. Such time-integral, dynamic neural models are becoming increasingly important for computationally constrained edge-devices that perform signal-processing tasks such as sensor fusion and embodied, closed-loop systems that are often subject to out-of-distribution inputs that must be handled gracefully. When evolution has the opportunity to tap into signals, timescales, and continuous values that are inaccessible to modern digital computing, we can expect challenges, but also tremendous opportunities. We hope that this paper and the associated toolkit will help push forward this field.

VIII. CONCLUSION

In this paper, we presented the *Bitstream Evolution* platform, which aims to advance research in the domain of intrinsic, bitstream-level, evolvable hardware. The platform is designed to facilitate community engagement and contributions by providing open-source software, accessible tutorials, and a

publicly available result repository. We strive to enable iterative enhancements by researchers globally to address the challenges and open questions in the field.

Using this platform we reported on findings from several experiments focused on the robustness and transferability of circuits evolved on modern hardware. These results show promising outcomes in the robustness and transferability of evolved circuits. Anyone can easily and productively contribute to this open domain of science through 1) re-running these same experiments with their personal hardware to help verify the reproducibility across a wider range of environments and 2) re-running these experiments with a different range of parameters. We provide tutorials on running first-time experiments in the appendix (A). However, this paper reports on a wider range of experimental setups, but all are capable of running out of the box.

Looking ahead, the platform sets the stage for exploring numerous avenues within evolvable hardware. Open questions about stability, transferability, robustness, and quality diversity remain, presenting exciting opportunities for further research. Our discussion of future directions includes enhancing the platform's capabilities to support more diverse experimental setups, which could significantly impact the adaptability and applicability of evolved circuits across different hardware configurations and operational conditions.

The collaborative and open-source nature of the *Bitstream Evolution* platform is designed to accelerate the pace of innovation in this field. By lowering the barriers to entry and encouraging global participation, we aim to foster a vibrant community of researchers dedicated to pushing the boundaries of what's possible with evolvable hardware.

APPENDIX A EXAMPLE EXPERIMENT TUTORIALS

This section walks through two sample experiments to illustrate how to use the system, along with the results from running these experiments. The goal of each experiment is to reproduce prior results quickly. In this section, we will walk through the key configuration parameters and explain how these will affect each experiment. Each experiment will contain a table of relevant configuration parameters (parameters that are excluded are irrelevant for the particular mode; must be configured per system, such as file paths, or are not the subject of the experiment and should be left at default values, such as the routing type). Additionally, each section will include plots generated from running the experiment to give an idea of what results should look like.

A. VARIANCE MAXIMIZATION EXPERIMENT

This first experiment uses the variance-maximization fitness function to evolve circuits that generate noisy signals, which is the easiest type of experiment for which to get rapid, positive results. The full list of relevant configuration settings is in Table 5. Additionally, the configuration file for the experiment outlined here is present in the *Bitstream Evolution* GitHub repository.

Next, we will provide explanations for the parameter values chosen and how those parameters operate, when not immediately clear. Whether or not to perform a mutation is evaluated on a per-bit basis, so a low mutation rate can still lead to many mutations in a single individual (with a bitstream of 1728 bits, this rate of 0.0021 results in an average of 3.6 bits flipped per individual each time that individual is mutated). Due to the nature of the stochastic fitness function (particularly when evolving intrinsically), high elitism is important, as a single broken connection can send a once-good circuit to a fitness of zero. Fitness-proportionate selection (FIT_PROP_SEL) is used, because it heavily favors high-fitness elites. We use the random initialization mode to evolve circuits completely from scratch.

Running this experiment ought to produce results similar to those in Figure 14. Note that these plots were generated by running the experiment for 500 generations rather than 200; however, all other parameters were the same. We primarily analyze the performance of the best circuit, which, as we can see, made jumps at around 100 and 200 generations before not making any additional improvements throughout the experiment. Diversity, as expected, drops significantly after the completely randomized population settles on a particular genotype.

In Figure 15, we can see the waveform generated by the highest-fitness individual in the final generation. As expected, it is fairly noisy, frequently leaping between the low and high voltage levels. The trigger voltage line refers to the threshold between what is considered a logic low and a logic high on the microcontroller. As we can see here, this circuit frequently crosses this threshold, and we would expect it to produce a large number of pulses using the pulse count fitness function.

The final plot that this experiment will generate is the voltage heatmap, shown in Figure 16. This plot shows the frequency of voltage levels for the best circuit in each generation. For example, for approximately the first 100 generations, readings of the voltage of the best circuit were between 0 and 0.5 volts, but typically somewhere in between. In later generations, it was more common to get readings at either 0 or 3.3 volts. The population made its first big jump around generation 100, which is consistent with the findings here.

TABLE 5. Table of Relevant Configuration Parameters

Parameter	Value
population_size	50
generations	200
target_fitness	IGNORE
fitness_func	VARIANCE
mutation_probability	0.0021
crossover_probability	0.7
elitism_fraction	0.1
selection	FIT_PROP_SEL
randomize_until	NO
init_mode	RANDOM
simulation_mode	FULLY_INTRINSIC

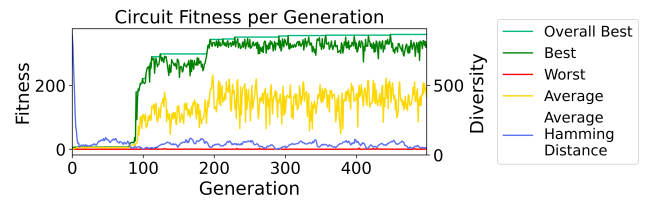


FIGURE 14. Fitness values and diversity for one run of the variance-maximization example experiment.

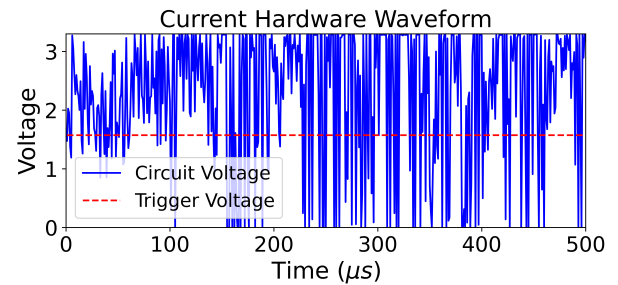


FIGURE 15. Waveform of best circuit in the variance-maximization experiment.

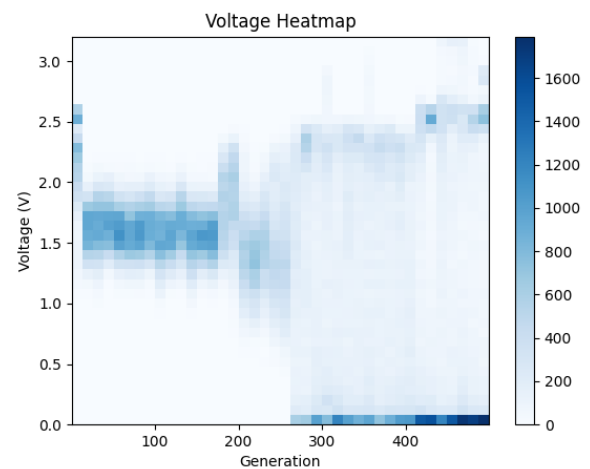


FIGURE 16. Heatmap for circuit voltage levels in the variance-maximization. Initially, circuits remain at a low voltage but evolve to alternate between high and low.

B. PULSE COUNT OSCILLATOR EXPERIMENT

This next experiment uses the pulse count fitness function to evolve oscillator circuits. Again, the full list of relevant configuration settings is provided in Table 6, and the configuration file for the experiment outlined here is present in the *Bitstream Evolution* GitHub repository.

We have found positive results in less than 500 generations. However, we have provided plots for this same experiment run for 500 generations and recommend running it for that long to achieve results similar to those in Section V-A.

Many of the parameters are the same as the variance-maximization experiment for the same reasons. We use the randomization feature to randomize bitstreams until we find a circuit that produces a non-zero number of pulses. Then, the evolutionary process begins; this ensures that you will start with an initial population of pulse-producing circuits for evolution to build off of. We have found that the tolerant and sensitive fitness functions tend to produce similar-caliber oscillators, so, because it has historically been used more often, we will be using the sensitive function. Additionally, because 40 kHz oscillators tend to be the most successful, we will set that as our target frequency.

In Figure 17, the pulse count experiment at first seems fairly unsuccessful, reaching the ideal fitness of 1 twice, but not consistently. However, it's important to note that this fitness function is fairly unforgiving. The jumps in best fitness up to values of 0.5 or 0.25 represent getting within 1 and 2 pulses of the target respectively, which is extremely difficult with such a high target fitness (being just 10 pulses off of a target of 40,000 results in a fitness of 0.1). We'll take a look at the pulse count graph to get more illuminating results.

The plot in Figure 18 displays data for the pulses counted, rather than the fitnesses. A red dashed line is drawn for the target frequency for reference. There are lines for the "best" pulse count for that generation (this is the count that is closest to the target), one each for the maximum and minimum pulses counted, and for the population average pulse count.

For this experiment, the maximum inconsistently oscillated between being very large or being near the target, though eventually settled down very close to the target fitness. The minimum did not fluctuate much at all, staying at 0, indicating that there was never a generation without a 0-pulse individual. There tends to be a higher frequency of low pulse counts versus high pulse counts (as seen below in Figure 19), which means the population average tends to be pulled below the target. Here, we can better analyze the performance of the best circuit than with raw fitness, and see that it clearly gets very close to the target frequency almost immediately and stays there for the remainder of the experiment.

Figure 19 shows the heatmap for pulse counts for each generation of the example oscillator experiment. We can see that the population tended to hover both around the target of 40,000 pulses and around 0 pulses. This indicates that many of the mutations simply stopped pulses from being generated at all, rather than slightly changing the number – demonstrating a key challenge of evolvable hardware.

TABLE 6. Table of Relevant Configuration Parameters

Parameter	Value
population_size	50
generations	500
target_fitness	IGNORE
fitness_func	SENSITIVE_PULSE_COUNT
desired_freq	40000
mutation_probability	0.0021
crossover_probability	0.7
num_samples	2
num_passes	1
elitism_fraction	0.1
selection	FIT_PROP_SEL
randomize_until	PULSE
randomize_threshold	0
init_mode	RANDOM
simulation_mode	FULLY_INTRINSIC

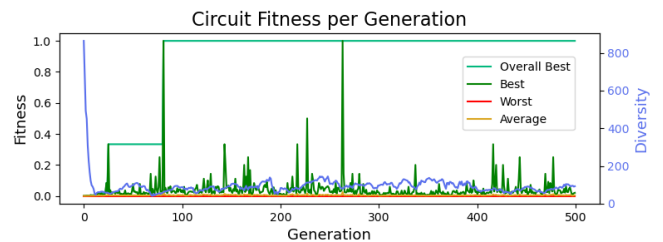


FIGURE 17. Results of running the pulse count example experiment

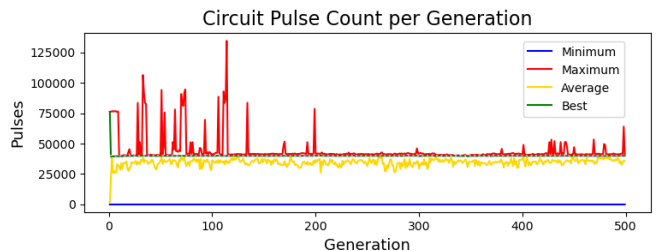


FIGURE 18. Plot of the best, average, and worst pulses counted for each generation

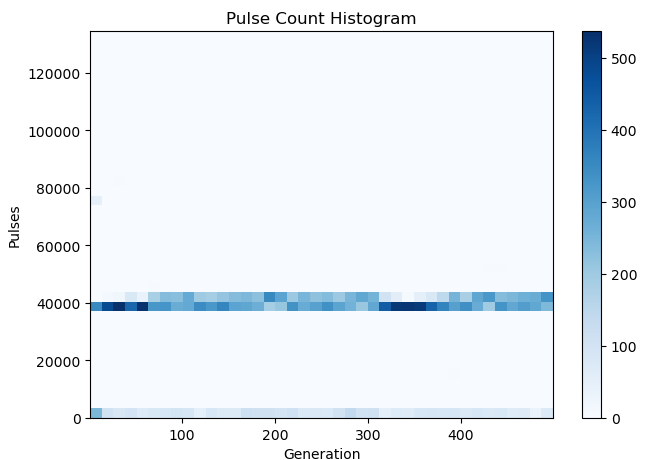


FIGURE 19. Heatmap for pulses counted in the example oscillator experiment

ACKNOWLEDGMENT

We would like to express our thanks to Thomas Bioren, Isaac Mierow, Vineet Ranade, Rebecca Testa, Kieran Hall, and Brooklyn Jennings for helping to conduct some of the experiments and ongoing work to improve this toolkit. ChatGPT was utilized to aid in rewording small sections of the paper.

REFERENCES

- [1] A. Thompson, "On the automatic design of robust electronics through artificial evolution," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 1478, pp. 13–24, 1998. [Online]. Available: <https://link.springer.com/chapter/10.1007/BFb0057603>
- [2] J. Hoffmann, C. Fritzsche, and M. Bogdan, "CoBEA: Framework for Evolving Hardware by Direct Manipulation of FPGA Bitstreams; CoBEA: Framework for Evolving Hardware by Direct Manipulation of FPGA Bitstreams." [Online]. Available: <https://doi.org/10.1145/3520304.3528821>
- [3] D. Whitley, J. Yoder, and N. Carpenter, "Resurrecting FPGA Intrinsic Analog Evolvable Hardware," in *The 2021 Conference on Artificial Life*. Cambridge, MA: MIT Press, 2021. [Online]. Available: https://direct.mit.edu/isal/article-abstract/doi/10.1162/isal_a_00448
- [4] M. Garvie, I. Flascher, A. Philippides, A. Thompson, and P. Husbands, "Evolved Transistor Array Robot Controllers," *Evolutionary Computation*, vol. 28, no. 4, pp. 677–708, 12 2020. [Online]. Available: <https://direct.mit.edu/evco/article/28/4/677-708/95003>
- [5] D.-C. Dang and P. K. Lehre, "Efficient Optimisation of Noisy Fitness Functions with Population-based Evolutionary Algorithms," in *Proceedings of the 2015 ACM Conference on Foundations of Genetic Algorithms XIII*. New York, NY, USA: ACM, 1 2015, pp. 62–68.
- [6] P. C. Haddow and A. M. Tyrrell, "Evolvable Hardware Challenges: Past, Present and the Path to a Promising Future," 2018, pp. 3–37. [Online]. Available: http://link.springer.com/10.1007/978-3-319-67997-6_1
- [7] F. Cancare, M. D. Santambrogio, and D. Sciuto, "A direct bitstream manipulation approach for Virtex4-based evolvable systems," in *Proceedings of 2010 IEEE International Symposium on Circuits and Systems*. IEEE, 5 2010, pp. 853–856. [Online]. Available: <http://ieeexplore.ieee.org/document/5537429/>
- [8] A. Thompson, "An evolved circuit, intrinsic in silicon, entwined with physics," in *Proceedings of the First International Conference on Evolvable Systems: From Biology to Hardware*, ser. ICES '96. Berlin, Heidelberg: Springer-Verlag, 1996, p. 390–405.
- [9] C. Wolf, "Project IceStorm." [Online]. Available: <https://clifford.at/icestorm>
- [10] I. Harvey, "The Microbial Genetic Algorithm," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Springer, Berlin, Heidelberg, 2011, vol. 5778 LNAI, no. PART 2, pp. 126–133. [Online]. Available: http://link.springer.com/10.1007/978-3-642-21314-4_16
- [11] J. Hoffmann, C. Fritzsche, and M. Bogdan, "CoBEA," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. New York, NY, USA: ACM, 7 2022, pp. 112–115. [Online]. Available: <https://dl.acm.org/doi/10.1145/3520304.3528821>
- [12] J. K. Pugh, L. B. Soros, and K. O. Stanley, "Quality diversity: A new frontier for evolutionary computation," *Frontiers Robotics AI*, vol. 3, no. JUL, p. 202845, 7 2016.
- [13] A. P. Shanthi and R. Parthasarathi, "Practical and scalable evolution of digital circuits," *Applied Soft Computing Journal*, vol. 9, no. 2, pp. 618–624, 3 2009.
- [14] J. Torresen, "A Scalable Approach to Evolvable Hardware," *Genetic Programming and Evolvable Machines* 2002 3:3, vol. 3, no. 3, pp. 259–282, 2002. [Online]. Available: <https://link.springer.com/article/10.1023/A:1020163325179>
- [15] T. G. W. Gordon, "Exploiting development to enhance the scalability of hardware evolution," Ph.D. dissertation, University College London, 7 2005.
- [16] D. Whitley, "Neuroevolving Electronic Dynamical Networks," 2024. [Online]. Available: <https://arxiv.org/abs/2404.04587>



ALLYN LOYD is currently pursuing a PhD in Industrial Engineering at the University of Houston. They are a recent graduate of Rose-Hulman Institute of Technology, with majors Computer Science and Math. They are also interested in operations research, transportation theory, and robust optimization.



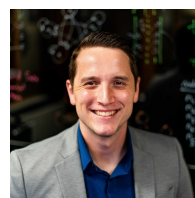
DANIEL GAULL is currently a software engineer at Tesla, Inc. He recently graduated from the Rose-Hulman Institute of Technology with majors in Computer Science and Software Engineering. He is curious about embedded systems, user-friendly design, and deep learning.



LOGAN MANTHEY is currently working as a FPGA engineer at Vivum AI. He is a recent graduate of Rose-Hulman Institute of Technology, with a Bachelors of Computer Engineering and Master of Engineering Management. His research interests include embedded systems, hardware acceleration, and neural network implementations on FPGAs.



NICKLAS CARPENTER graduated Rose-Hulman Institute of Technology with a Bachelor's of Science in Computer Engineering and a minor in Computer Science. He is currently pursuing a PhD in Computer Science at the University of Arizona researching abstract interpretation, program analysis, and computational complexity theory.



DEREK WHITLEY holds a dual PhD in complex systems and cognitive science from Indiana University, is the CTO and co-founder of Vivum AI, and is Visiting Assistant Professor at Rose-Hulman Institute of Technology in Computer Science and Software Engineering. His research interests focus on the application of evolvable hardware, robotics, and neuroethology to both industry and scientific enterprises.



JASON A. YODER earned a dual PhD in computer science and cognitive science from Indiana University in 2018. He is currently an Associate Professor at Rose-Hulman Institute of Technology in Computer Science and Software Engineering. His research interests focus on evolutionary and adaptive systems that include elements of evolution, development, and/or learning. He enjoys teaching courses such as bio-inspired AI and object-oriented software development.